

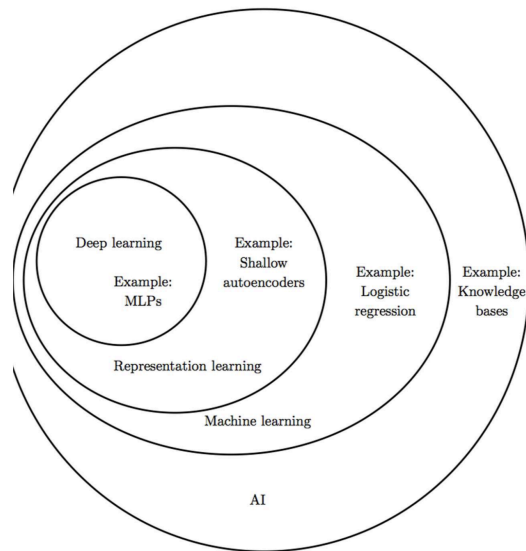
CS324 Final Review Notes

Gong Linghu

SID: 12110631

History of Deep Learning

AI Categories



- Knowledge Bases: 使用硬编码的推理规则
- Machine Learning: 机器通过在原始数据中寻找模式（pattern）来学习
- Representation Learning: 自动编码器在保护信息并且添加良好特性的基础上，学习如何编码和解码数据
- Deep Learning: 特征有时较为复杂，难以从原始数据中提取。需要有层次地提取，从浅层学习到深度学习

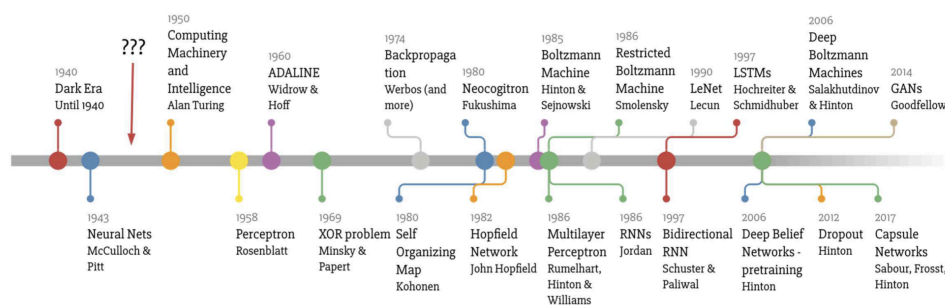
Machine Learning vs Deep Learning

深度学习相比机器学习，同时还会学习如何表示数据。

Genres of Artificial Intelligence

1. **Symbolism**: 基于符号操作和逻辑推理构建 AI 系统。专家系统、逻辑推理、规则系统。
2. **Connectionism**: 通过模拟神经网络实现智能行为。人工神经网络、深度学习。“Cybernetics”。
3. **Behaviorism**: 通过试验和反馈学习优化行为。强化学习。

Timeline of the Development of Deep Learning



- 1943~1960: Cybernetics
- 1969: XOR problem

需要大致了解以上时间线与其中各项的区别。

Perceptron

Single Layer Perceptron

$$\hat{y} = \sigma(w^T x + b)$$

Training Algorithm

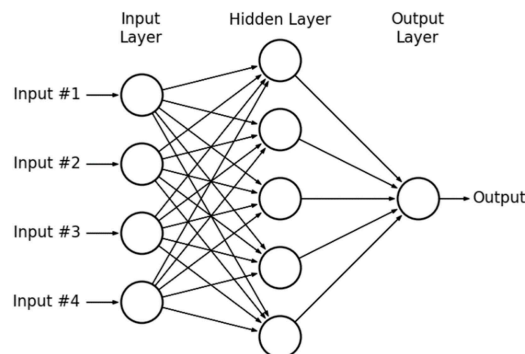
Given training set $D = \{(x_i, y_i)\}, x_i \in \mathbb{R}^n, y_i \in \{-1, 1\}$.

1. 初始化 $w = 0$
2. 对于每一个 epoch
 1. 随机打乱数据
 2. 对于每一个样本 $(x_i, y_i) \in D$, 如果 $y_i \hat{y} \leq 0$, $w \leftarrow w + r y_i x_i$
3. 返回 w

Multi-Layer Perceptron

单层感知机无法解决 XOR 问题，因为 XOR 问题不是线性可分的。于是引入非线性变化，使得数据在新的空间中线性可分。可以采用的方法

- Kernel methods
- Deep learning: deeper networks



Gradients

Loss Functions

1. Cross entropy

$$L(\tilde{y}) = - \sum_{i=1}^C y_i \log(\tilde{y}_i)$$

$$\frac{\partial L}{\partial \tilde{y}_i} = -\frac{y_i}{\tilde{y}_i}$$

1. Euclidean loss: commonly used in regression

$$L(\tilde{y}) = \frac{1}{2} \|y_i - \tilde{y}_i\|^2$$

$$\frac{\partial L}{\partial \tilde{y}} = \tilde{y} - y$$

Activation Functions

1. Sigmoid

$$a = \sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\frac{\partial a}{\partial x} = \sigma(x)(1 - \sigma(x))$$

2. tanh

$$a = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\frac{\partial a}{\partial x} = 1 - \tanh^2(x)$$

3. ReLU

$$a = h(x) = \max(x, 0)$$

$$\frac{\partial a}{\partial x} = \begin{cases} 0 & \text{if } x \leq 0 \\ 1 & \text{if } x > 0 \end{cases}$$

- 在 ReLU 引入前，Sigmoid 和 tanh 都是非常流行的激活函数
- 非常容易饱和 (saturate)，这可以被解释成网络单元容易 over confident，导致停止学习
- 小的梯度很容易与其他的小梯度相乘 → 梯度消失
- tanh 更适合在隐藏层中使用
- 还有许多其他的单元。例如 hinge loss, dropout, L1-regularization
 - Hinge loss: $L = \max(0, 1 - y \cdot f(x))$ ，最大化分类间隔，对训练样本的错分类和边界内的样本施加惩罚

Forward and Backward Propagation

Gradient Descent

$$w^{(t+1)} = w^{(t)} - \eta_t \nabla_{w^{(t)}} L$$

where $\nabla_{w^{(t)}} L$ is the gradient w.r.t. parameters.

For the l -th layer,

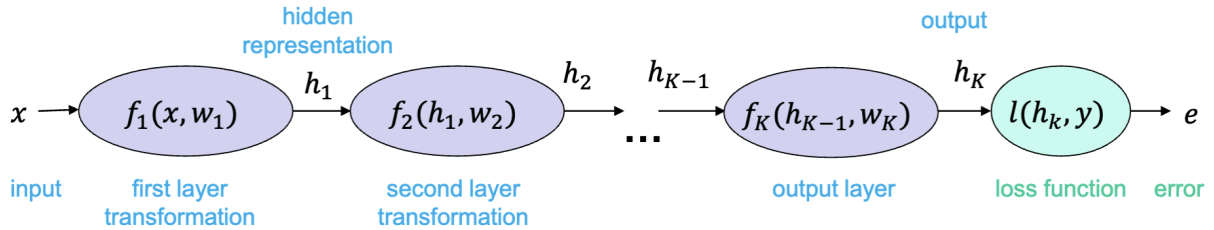
$$\frac{\partial L}{\partial w^l} = \frac{\partial L}{\partial a^L} \frac{\partial a^L}{\partial a^{L-1}} \cdots \frac{\partial a^l}{\partial w^l}$$

$$= \left(\frac{\partial a^l}{\partial w^l} \right)^T \frac{\partial L}{\partial a^l}$$

where

$$\begin{aligned}\frac{\partial L}{\partial a^l} &= \left(\frac{\partial a^{l+1}}{\partial a^l} \right)^T \frac{\partial L}{\partial a^{l+1}} \\ &= \left(\frac{\partial a^{l+1}}{\partial x^{l+1}} \right)^T \frac{\partial L}{\partial a^{l+1}}\end{aligned}$$

Computational Graph



找到 error 相对于每一层的参数 w_k 的梯度 $\frac{\partial e}{\partial w_k}$ 更新参数

$$w_k \leftarrow w_k - \eta \frac{\partial e}{\partial w_k}$$

对于 $\frac{\partial e}{\partial w_k}$, 有

$$\frac{\partial e}{\partial w_k} = \frac{\partial e}{\partial h_K} \frac{\partial h_K}{\partial h_{K-1}} \cdots \frac{\partial h_{k+1}}{\partial h_k} \frac{\partial h_k}{\partial w_k}$$

其中

- $\frac{\partial e}{\partial h_k}$ 被称为上游梯度 (Upstream gradient)
- $\frac{\partial h_k}{\partial w_k}$ 及 $\frac{\partial h_k}{\partial h_{k-1}}$ 被称为局部梯度 (Local gradient)

于是, 第 k 层的更新梯度为

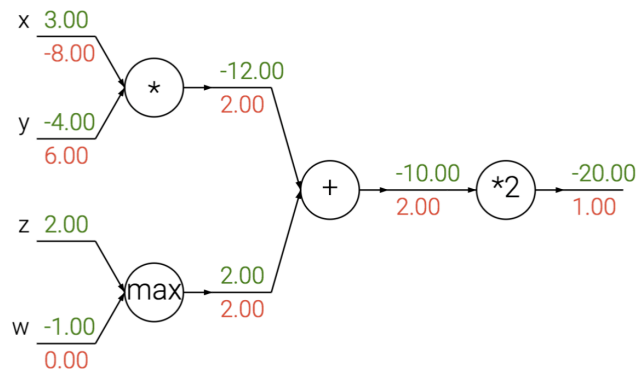
$$\frac{\partial e}{\partial w_k} = \frac{\partial e}{\partial h_k} \frac{\partial h_k}{\partial w_k}$$

而上游梯度 (相对 f_{k-1})

$$\frac{\partial e}{\partial h_{k-1}} = \frac{\partial e}{\partial h_k} \frac{\partial h_k}{\partial h_{k-1}}$$

会在计算图上继续传播。

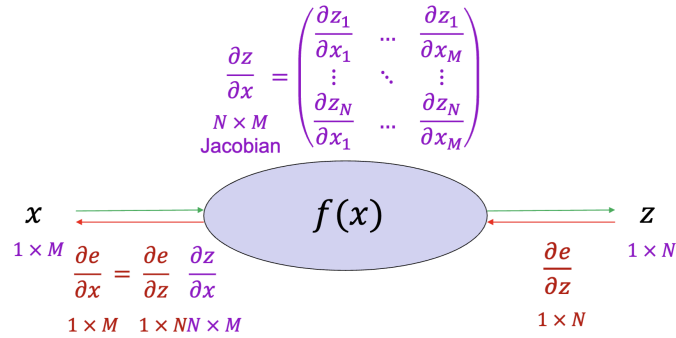
Patterns in Gradient Flow



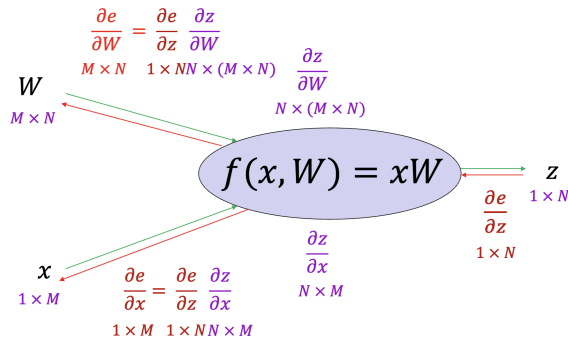
- Add gate: “gradient distributor” 平等地分配梯度
- Multiply gate: “gradient switcher” 梯度交叉相乘流向不同的方向
- Max gate: “gradient router” 选择梯度最大的路径

Vectorized Computing

无参的计算与标量的类似，导出 Jacobian 矩阵后进行计算即可。



有可学习参数时，情况变得较为复杂。



Optimization

Gradient Descent

$$w^{(t+1)} = w^{(t)} - \eta_t \nabla_{w^{(t)}} L$$

Batch Gradient Descent

m 个训练样本作为一批，计算平均梯度

$$w^{(t+1)} = w^{(t)} - \eta_t \nabla_{w^{(t)}} L$$

$$\nabla_{w^{(t)}} L = \frac{1}{m} \sum_{i=1}^m \nabla_{w^{(t)}} L(w; x_i, y_i)$$

注意到，这里的梯度 $\nabla_{w^{(t)}} L$ 实则只是样本梯度，知道真实数据分布的情况下与真实梯度不一样。

在实际情况下，我们只需要计算平均损失来计算梯度即可

$$L = \frac{1}{m} \sum_{i=1}^m L(x_i, y_i)$$

Advantages

1. Batch GD 可以使用基于二阶导数的加速方法 (Hessian 矩阵)
 - Batch GD 每次迭代中使用整个数据集计算梯度和损失函数值, 从而为这些技术提供了可靠且一致的全局信息
 - Hessian 矩阵描述了损失函数的曲率信息。通过直接利用目标函数的二次结构, 能够在少量迭代中找到最优解。
2. 可以对收敛速度进行简单的理论分析。

Disadvantages

1. 数据集过大时, 难以进行完整的梯度计算
2. 损失函数可能是高度非凸, 并且在高维度

Stochastic Gradient Descent

一个训练样本更新一次权重。

$$w^{(t+1)} = w^{(t)} - \eta_t \nabla_{w^{(t)}} L(w; x_i, y_i)$$

Advantages

1. 比 Gradient Descent 收敛更快。从第一个样本开始改进模型, 并且考虑完整的训练数据集可能会冗余。
2. 随机性可以帮助解决过拟合, 提升准确度。
3. 适合用于会随着时间改变的数据集。

Disadvantages

1. 在传统的 Batch Gradient Descent 中, 梯度是基于整个数据集计算的, 是目标函数梯度的准确值。而在 SGD 中, 梯度估计只使用一个样本, 会导致不准确。
 - 但是实际中反而是优势, 因为噪声可以帮助模型解决过拟合问题
2. 不能大量并行计算。

Mini-batch Gradient Descent

使用多个数据样本来计算梯度, 而不是整个数据集。

$$w^{(t+1)} = w^{(t)} - \frac{\eta_t}{|B|} \sum_{b \in B} \nabla_{w^{(t)}} L(w; b)$$

- B 是一个小的数据集样本
- 一个更加通用的 SGD 版本, 所以通常被称为 SGD

Momentum

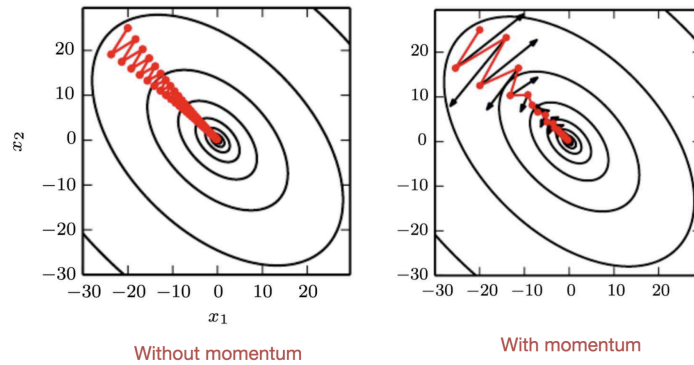
- 使用速度 (velocity) 跟踪过去梯度的指数加权平均值, 累加过去的梯度
- 抑制梯度震荡, 让梯度更加平滑 (鲁棒), 加速收敛

$$\text{Compute gradient estimate: } g \leftarrow \frac{1}{m} \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$$

$$\text{Compute velocity: } v \leftarrow \alpha v + \varepsilon g$$

$$\text{Apply update: } \theta \leftarrow \theta + v$$

- 通常 α 在开始的时候较低 (为 0.5, 0.9 或者 0.99), 随着时间增加



Nesterov Momentum

- 使用“未来”的梯度更新
- 通过先前的速度更新来调整梯度，而不是直接使用当前的梯度，最后再应用一次速度

Apply interim update : $\tilde{\theta} \leftarrow \theta + \alpha v$

Compute gradient estimate: $g \leftarrow \frac{1}{m} \nabla_{\theta} L(f(x^{(i)}; \tilde{\theta}), y^{(i)})$

Compute velocity: $v \leftarrow \alpha v - \varepsilon g$

Apply update : $\theta \leftarrow \theta + v$

Adaptive Learning Rates

AdaGrad

- 通过与过去所有梯度平方值之和的平方根成反比，调整模型参数的学习率
- 损失偏导数值大（小）的参数学习率下降（上升），使得在较为平滑的方向上收敛较快
- 但是会导致学习率过早下降，导致收敛速度过慢

RMSProp

- 通过指数加权平均（EMA，exponential moving average）的方式，调整学习率
- AdaGrad 对于凸函数效果好，但是对于非凸函数效果不佳。RMSProp 通过 EMA 的方式，使得学习率更加平滑，适用于非凸函数

Adam

- 结合了 Momentum 和 RMSProp 的优点
- 使用平方的梯度来调整学习率，同时使用梯度的一阶矩估计和二阶矩估计（uncentered variance）来调整学习率

Second Order Optimization

- 二阶优化方法通过使用 Hessian 矩阵来调整学习率
- 基于牛顿法

Regularization

- 深度学习网络通常具有上百万的参数，比较容易过拟合
- 通过添加正则化项来限制模型的复杂度，减少模型容量（reduce the model capacity）

$$w^* = \arg \min_{x, y} \sum^n L(w_1, \dots, w_L; x, y) + \lambda \Omega(\theta)$$

- 通常的方法有 L1 和 L2 正则化，以及 Dropout

L1 Regularization

L1 正则化强制使得参数稀疏，比如让更多的参数变成 0（i.e. 删除连接，让神经元失去活性）

$$w^* = \arg \min_{x,y} \sum^n L(w_1, \dots, w_L; x, y) + \lambda \sum_l |w_l|$$
$$w_l = w_l - \lambda \eta \frac{w_l}{|w_l|} - \eta \nabla_{w_l} L$$

通常来说，具有特征选择作用，也就是说我们需要保留哪些输入数据的哪些特征。

L2 Regularization

- 对于回归任务来说更加平滑，能够让参数更接近原始的数据分布
- 具有令好的分析特性，便于求导以计算梯度
- 通常使用的 λ 值为 10^{-1} 或者 10^{-2}

$$w^* = \arg \min_{x,y} \sum^n L(w_1, \dots, w_L; x, y) + \lambda \sum_l \|w_l\|_2^2$$
$$w_l = (1 - \lambda \eta) w_l - \eta \nabla_{w_l} L$$

Dataset Augmentation

- 使用更多的训练数据来对抗过拟合
- 通过对训练数据进行变换，增加数据的多样性

Early Stopping

- 在验证集的损失函数上达到最小值时停止训练

Dropout

- 在训练过程中，随机地将一些神经元的输出设置为 0
- 能够加速训练、减少过拟合、使得单元变得更加鲁棒
- 实则作为一种集成学习方法，通过多次训练不同的子网络，最后将结果取平均

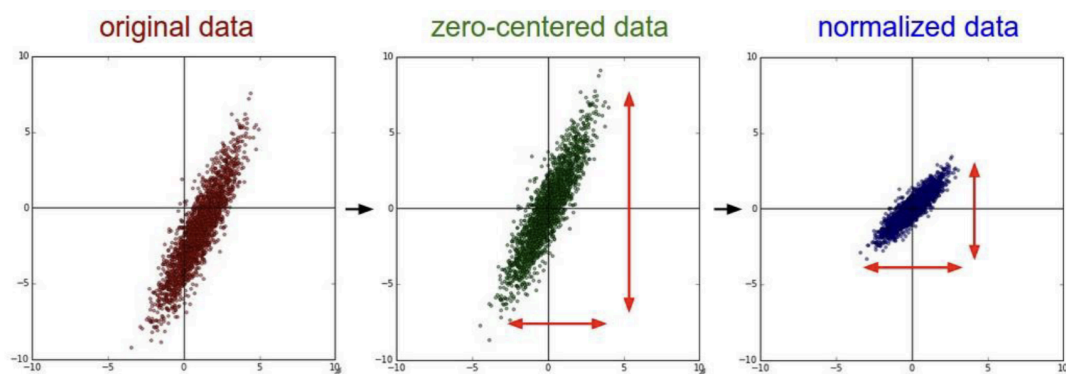
Weight Initialization

- 深度学习极强的依赖原始点，重要的原理：避免对称性
 - 如果两个单元具有同样的结构，那么它们的梯度将会相同，导致它们的权重在训练过程中保持相同的值，就不能够学习
 - 于是最好不要初始化为相同的值，可以使用高斯分布或者均匀分布初始化权重
- 同时注意以下一些点
 - 很大的权重具有很强的对称性，很小的权重一样也会导致一些问题
 - 我们还同时希望在输入和输出具有相同的方差

Normalization

- 激活函数通常在 0 周围表现最好，所以我们需要对数据进行标准化
- 这是一件好事，因为它可以帮助我们避免饱和，从而避免梯度消失
- 同时，我们也喜欢单侧饱和度，因为它有助于避免噪音造成的差异

Data Preprocess (Input Normalization)



Batch Normalization

输入网络层的数据需要满足

- 均值为 0（数据预处理的时候已经完成）
- 在时间和空间上的方差均匀（mini batches）

于是我们可以在每层的激活函数后，进行标准化

$$Z = XW$$

$$\tilde{Z} = Z - E[Z]$$

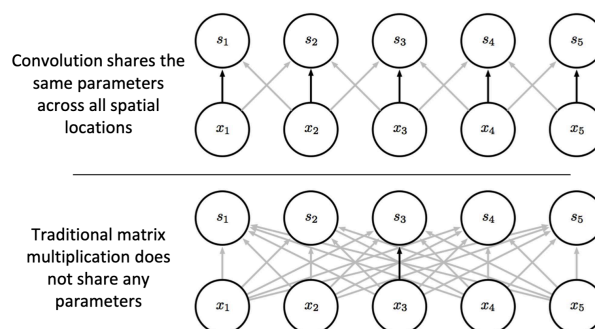
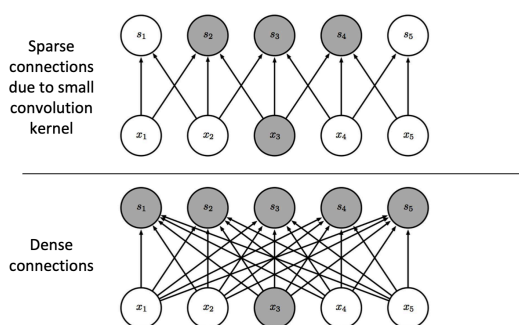
$$\hat{Z} = \frac{\tilde{Z}}{\sqrt{\text{Var}[Z] + \epsilon}}$$

$$H = \max\{0, \gamma\hat{Z} + \beta\}$$

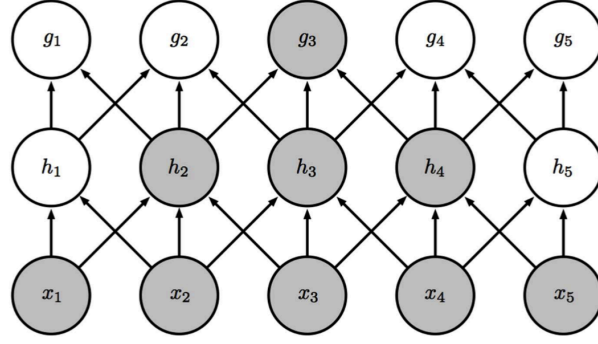
Convolutional Neural Networks

Motivation & Advantages

- 通过卷积核来保留空间结构
- 能够通过稀疏连接、权重共享来 scale up 以处理非常大的输入



- 通过池化，对局部的变化具有鲁棒性
- 扩大的感受野（Receptive fields）



Dimension After Convolutional

$$h_{\text{out}} = \frac{h_{\text{in}} - h_f + 2 \times h_p}{s} + 1$$

$$w_{\text{out}} = \frac{w_{\text{in}} - w_f + 2 \times w_p}{s} + 1$$

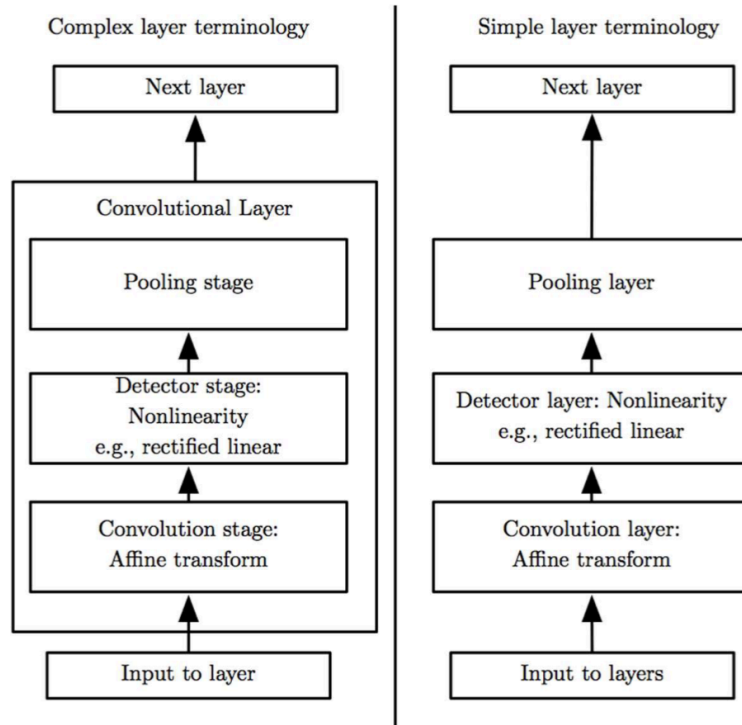
$$d_{\text{out}} = n_f$$

$$d_f = d_{\text{in}}$$

其中,

- $h_{\text{in}}, h_{\text{out}}, h_f, p$: 输入、输出、卷积核
- $w_{\text{in}}, w_{\text{out}}, w_f, p$: 输入、输出、卷积核
- $d_{\text{in}}, d_{\text{out}}, d_f$: 输入、输出、卷积核的维度
- s : 步长
- p : padding
- n_f : 卷积核的数量

CNN Components



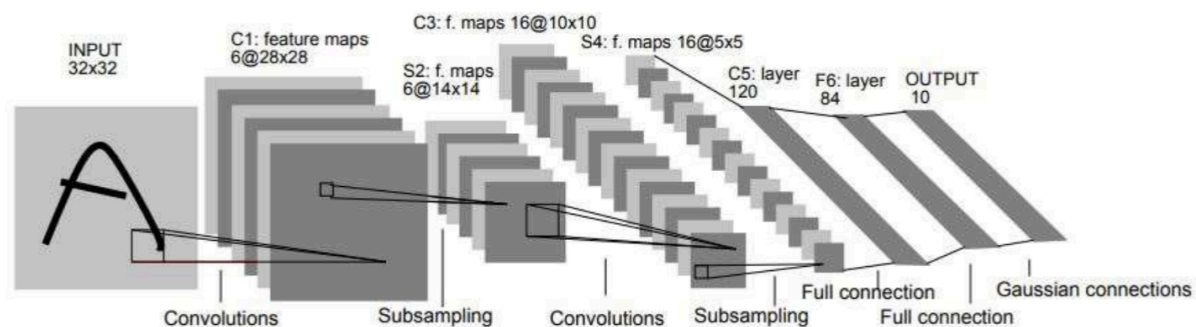
Pooling

- 将多个值聚合成一个单独的值
 - 能够减小层的大小，加速运算（下采样）
 - 保持对于下一层最重要的信息
 - 对于微小的变化具有不变性

Variants of CNN

LeNet-5

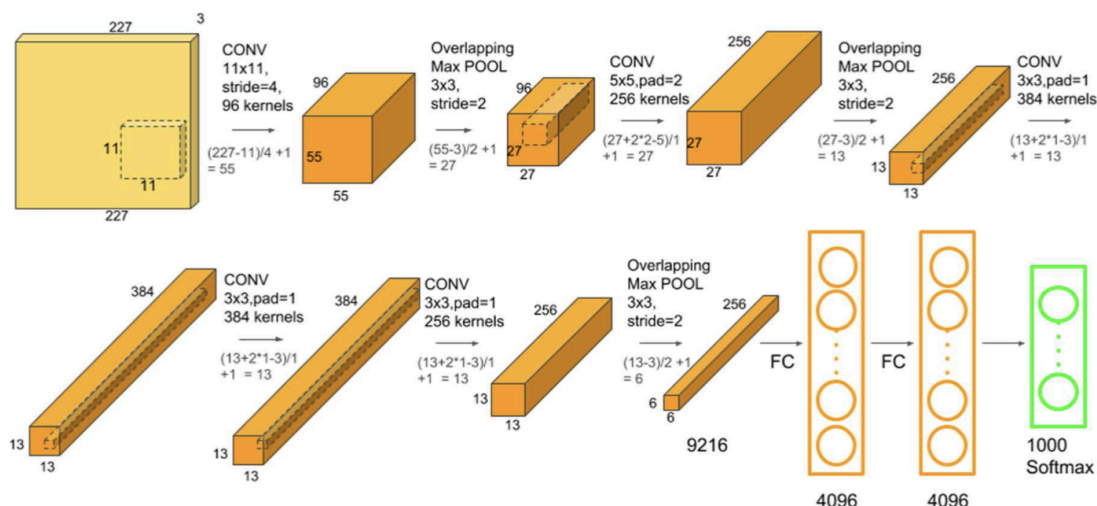
LeCun 提出的 7 层卷积神经网络，用于识别手写数字。但是由于算力限制



Original Image published in [LeCun et al., 1998]

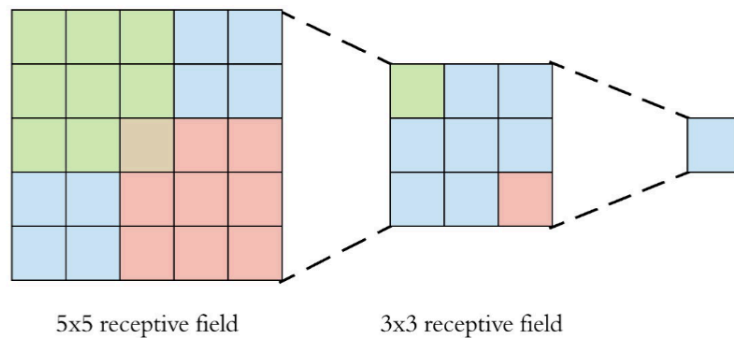
AlexNet

- 使用 ReLU，而不是 tanh 作为激活函数
- 使用了 Dropout 来减少过拟合
- 5 个卷积层，3 个全连接层



VGG (Visual Geometry Group)

- 16 个卷积层，使用 3*3 的卷积核
- 随着网络深度的增加，卷积核的数量增加
- 卷积核以 2 个或者 3 个一组堆积，2 个一组堆积的 3*3 卷积核具有 5*5 的感受野
 - 堆叠小的卷积核可以应用更多的 ReLU，产生更多的非线性，以训练更加强大的模型
 - 更少的参数，三个堆叠的卷积核只用 27 个参数，而相同感受野的 7*7 卷积核用 49 个



GoogLeNet (Inception Network V1)

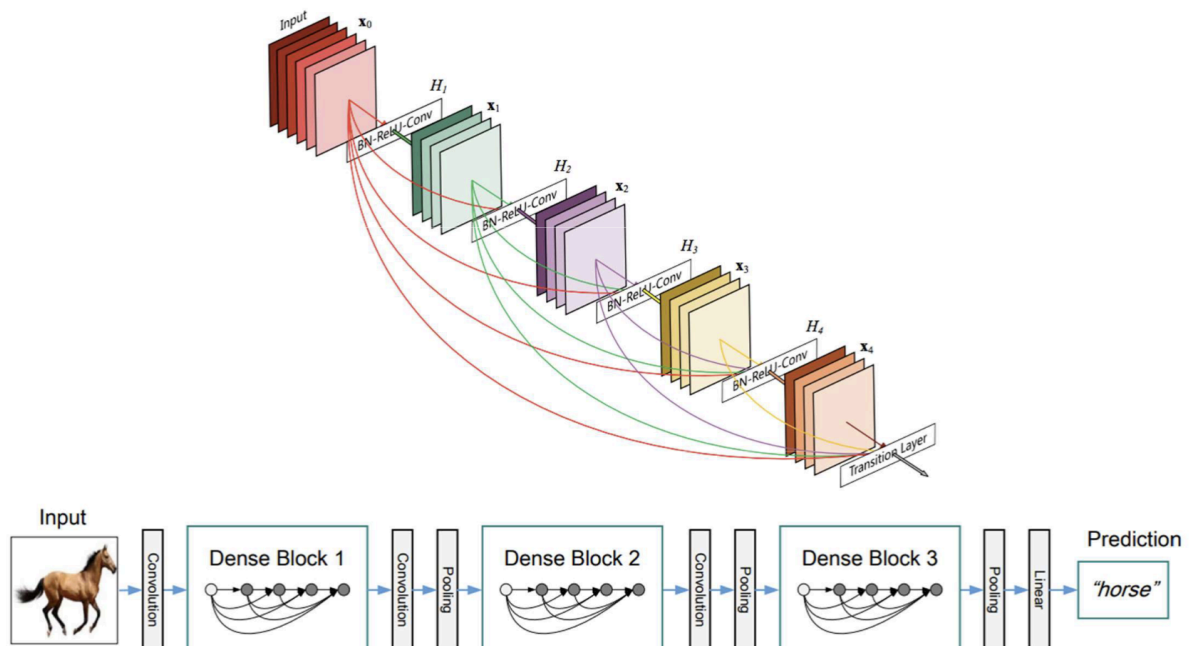
- 让网络更宽而不是更深，在相同的深度，使用多个不同大小的卷积核
- 使用 padding 让输出和输入的大小相同
- 使用 1*1 的卷积核来限制通道数量
- 使用 auxiliary classifiers 来解决梯度消失问题

ResNet

- 使用残差块 (Residual Block) 来解决梯度消失问题
 - 引入了跳跃连接 (skip connection)，将输入直接加到输出上
 - 使网络能够非常简单的学习恒等函数

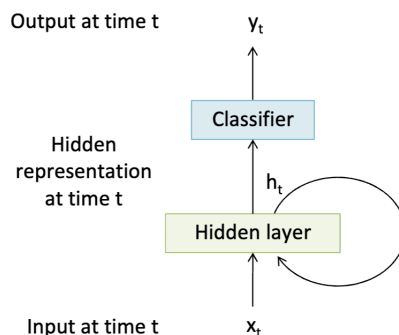
DenseNet

- 让神经网络的参数更加高效
 - 减小卷积核大小，重构卷积核
 - 在昂贵的操作前使用 1*1 卷积来减少特征图谱的数量
 - 最小化对于 FC 层的依赖
- 逐步降低空间分辨率，在每一级分辨率内多次复制特定“区块”
- 使用跳跃链接，或者在网络中构造多个冗余路径



Recurrent Neural Network

RNN



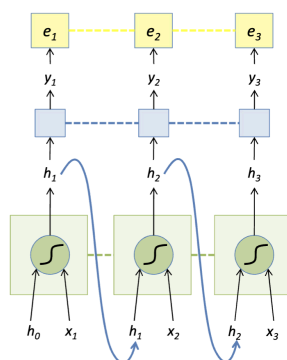
$$h_t = f_{W(x_t, h_{t-1})}$$

朴素的 RNN 单元

$$h_t = f_{W(x_t, h_{t-1})} = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

$$\tanh(a) = \frac{e^a - e^{-1}}{e^a + e^{-a}} = 2\sigma(2a) - 1$$

RNN 前向传播



$$e_t = -\log(y_t(GT_t))$$

$$y_t = \text{softmax}(W_y h_t)$$

$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

Backpropagation Through Timeline (BPTT)

- 训练 RNN 最常见的方式
- 展开 forward pass 后的网络会被当作一个巨大的、以整个时间序列为输入的前馈网络
- 对于每一个展开后的网络，权重会被计算一次以用于更新，此后梯度的和（或者平均梯度）会被用来更新 RNN 的权重
- 在实际中，使用 truncated BPTT：前向传播 k_1 次，反向传播 k_2 次

$$h_t = \tanh(W_x x_t + W_h h_{t-1})$$

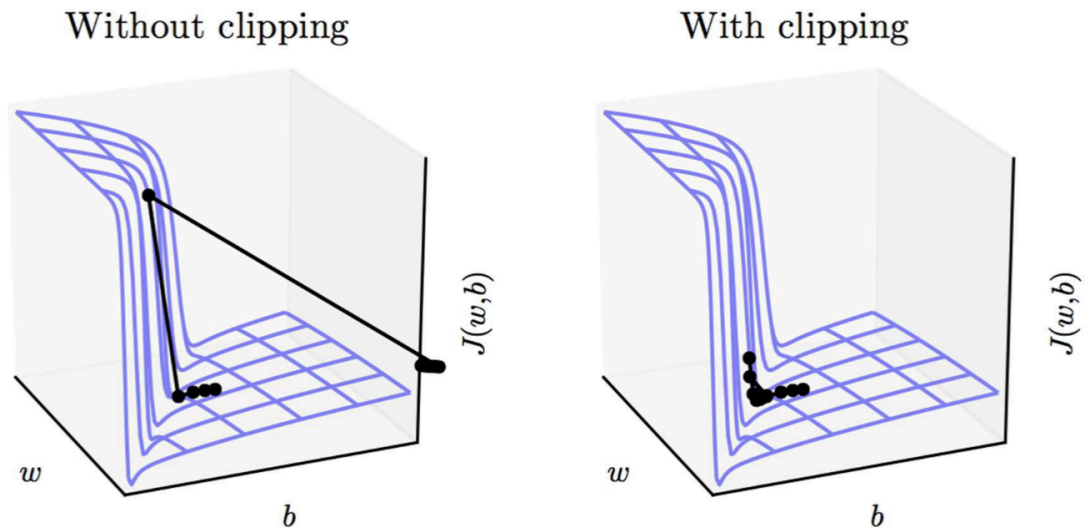
$$\frac{\partial e}{\partial W_h} = \frac{\partial e}{\partial h_t} \odot (1 - \tanh^2(W_x x_t + W_h h_t - 1)) h_{t-1}^T$$

$$\frac{\partial e}{\partial W_h} = \frac{\partial e}{\partial h_t} \odot (1 - \tanh^2(W_x x_t + W_h h_t - 1)) x_t^T$$

$$\frac{\partial e}{\partial h_{t-1}} = W_h^T (1 - \tanh^2(W_x x_t + W_h h_t - 1)) \odot \frac{\partial e}{\partial h_t}$$

梯度消失 & 梯度爆炸

- 由于 $\tanh$ 激活函数在变亮较大的时候，会给予非常小的梯度
- 在 $k \ll n$ 情况下考虑 $\frac{\partial e_n}{\partial h_k}$ ，如果 W_h 中的最大特征值（参考长距离依赖中的公式推导）比 1 小，那么梯度就会消失
- 对于非常非常深的网络，我们需要将非常多不同的梯度相乘，于是在相对较长的序列就会有梯度消失或者梯度爆炸的问题
- 可以使用梯度裁剪（gradient clipping）来处理梯度爆炸



长距离依赖

考虑某时刻状态对于初始状态的依赖

$$h^{(t)} = (W^t)^T h^{(0)}$$

进行奇异值分解

$$h^{(t)} = Q^T \Lambda^t Q h^{(0)}$$

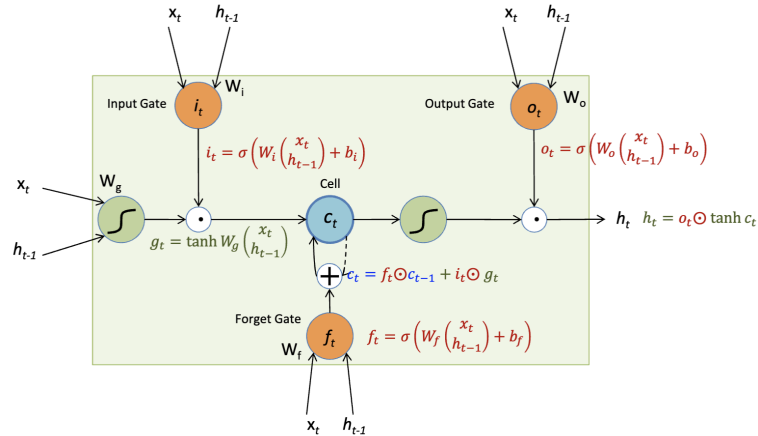
$$W = Q \Lambda Q^T$$

于是特征值比 1 小梯度就会消失，比 1 大就会爆炸。

- 这是由 RNN 的共享权重导致的，我们不能直接避免，因此学习长距离依赖是必要的
- 通常来说，训练 > 10 长度的序列会导致 RNN 训练缓慢和梯度消失

Long Short-Term Memory

- LSTM 的主要思想是，添加一个 memory cell，不直接参与矩阵乘法来避免梯度的问题



LSTM 的门

- Input gate: $i_t = \sigma(W_i(x_t, h_{t-1}) + b_i)$, 会与 $\tanh$ 激活函数的门在更新 cell 前逐元素相乘
- Output gate: $o_t = \sigma(W_o(x_t, h_{t-1}) + b_o)$, 会与 cell 在输出前与 $\tanh c_t$ 逐元素相乘
- Forget gate: $f_t = \sigma(W_f(x_t, h_{t-1}) + b_f)$, 用于选择性更新 cell

更新方式

$$\begin{pmatrix} g_t \\ i_t \\ f_t \\ o_t \end{pmatrix} = \begin{pmatrix} \tanh \\ \sigma \\ \sigma \\ \sigma \end{pmatrix} \begin{pmatrix} W_g & W_i & W_f & W_o \end{pmatrix} \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

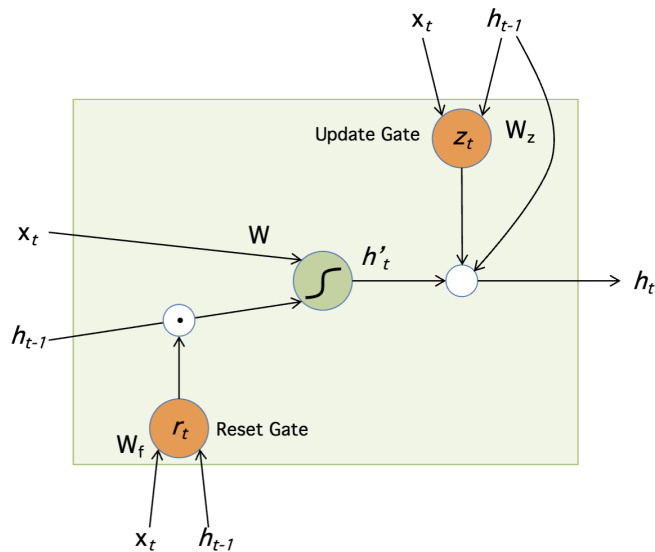
$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

$$h_t = o_t \odot c_t$$

- 由于 c_t 流向 c_{t-1} 的梯度只涉及逐元素相乘或者相乘，没有或者乘法或者 $\tanh$ 激活函数，不会产生梯度消失

Gated Recurrent Unit

- 放弃单独的存储状态的 cell，将遗忘门和输出门合并成更新门



$$r_t = \sigma \left(W_r \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix} + b_t \right)$$

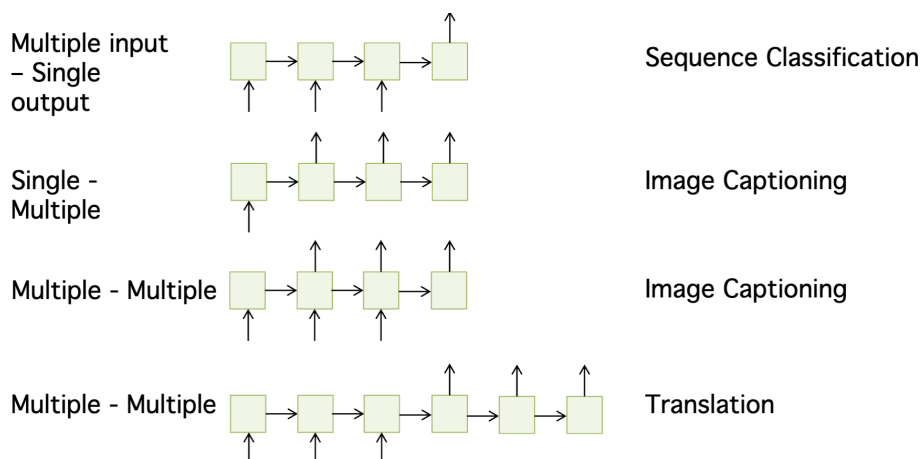
$$h'_t = \tanh W \begin{pmatrix} x_t \\ r_t \odot h_{t-1} \end{pmatrix}$$

$$z_t = \sigma \left(W_z \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix} + b_z \right)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot h'_t$$

Other RNNs

- **Multi-layer RNN:** 设计多个隐藏状态，使用层间或者时间轴上的跳跃链接。
- **Bi-directional RNN:** 同时正向和反向的处理序列，在语音识别上很常用



Manifold Learning

流形假设 (Manifold Hypothesis): 真实世界数据的分布在低维度、非线性的流形上。也就是说，数据在空间中一块非线性区域中高度集中。

Principal Component Analysis (PCA, 主成分分析)

目标：找到最佳的能够解释数据分布方差的方向

给定 $\mathbb{R}^d$ 中的数据点 $x_1, \dots, x_N$, 选择一个单位向量 $u \in \mathbb{R}^d$, 能最好的描述数据方差，于是我们就可以通过投影映射新的数据特征

$$u(x_i) = u^T(x_i - \mu)$$

于是就是最大化

$$\frac{1}{N} \sum_{\{i=1\}}^N u^T(x_i - \mu)(u^T(x_i - \mu))^T$$

即

$$u^T \Sigma u$$

其中 Σ 为数据的协方差矩阵。

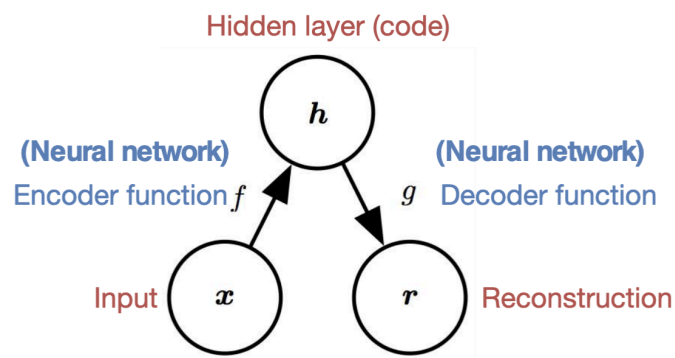
从几何意义上来说，就是最小化数据到单位向量上的投影。方向即为特征向量。

实际问题

- 协方差矩阵很大，而且通常来说样本数量要小于协方差矩阵的维度
- 可以通过以下方式处理
 - 中心化训练数据 X
 - 找到 X^T 的特征向量 u ，而不是 $X^T X$ ，然后 $X^T u$ 就是协方差 $X^T X$ 的特征向量，最后可能需要标准化为单位向量
- 由于 PCA 只能做正交变换，我们只能通过 PCA 有限的捕捉数据在原始空间中的结构

Autoencoders

- An autoencoder is neural networks that is trained to copy its input to its output.
- 通过学习非线性变换泛化了 PCA
- 通过流形假设，我们希望编码相比输入具有更低的维度，通过学习数据的低维表示捕捉输入的重要特征



- 目标：最小化重构损失

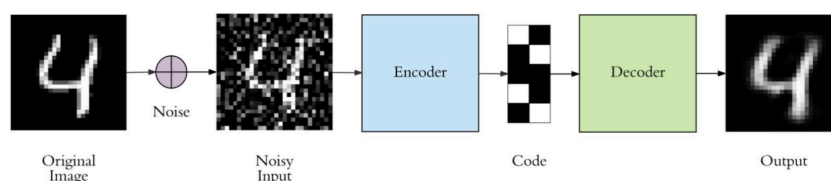
$$\frac{1}{N} \sum_{n=1}^N L(x_n, r_n) = \frac{1}{N} \sum_{n=1}^N L(x_n, g(f(x_n)))$$

自动编码器的概念

- Undercomplete autoencoders
 - 编码具有相比输入更低的维度
 - f 或者 g 具有较少的参数量
- Overcomplete Autoencoders
 - 编码具有更高的维度
 - 必须被正则化以对抗过拟合
 - 希望模型除了复制输入输出以外还具有一些有用的特性

自动编码器的变种

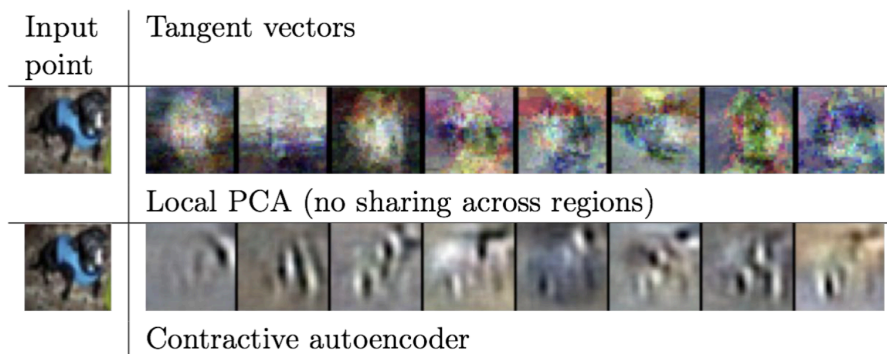
- 稀疏自动编码器 (Sparse autoencoders): 通过添加一个成本项 $\lambda \|h\|_1$ 来限制自动编码器的容量，该成本项惩罚代码变大，通常用于提取鲁棒特征或降低输入数据的维度
- 去噪自动编码器 (Denoising autoencoders): 通过引入噪声 $C(x|x)$ (可以是高斯噪声，也可以是 Dropout) 来学习去噪的映射，强制学习更加鲁棒的表示



- 收缩自动编码器 (Contractive autoencoders) : 更加显式地学习流形, 通过惩罚编码器 Jacobian 矩阵的 Frobenius 范数, 强迫编码器学习在训练样本周围变化不大的表示

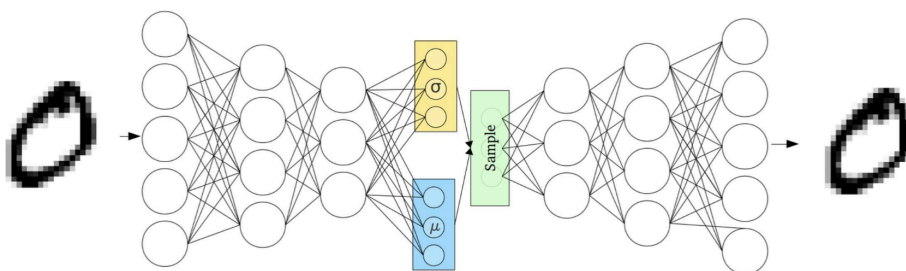
$$\frac{1}{N} \sum_{n=1}^N L(x_n, g(f(x_n))) + \Omega(h, x)$$

$$\Omega(h, x) = \lambda \left\| \frac{\partial f(x)}{\partial x} \right\|_F^2$$

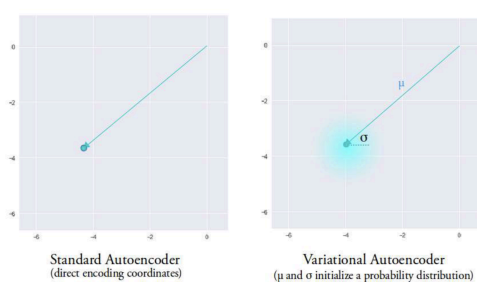


Variational Autoencoders

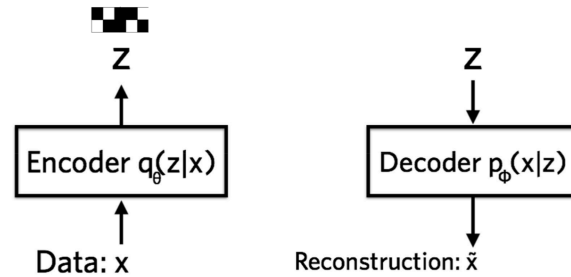
如果我们需要“生成”数据, 便需要 z 的数据分布, 那么只有 AE 无法达成。



- 与 AE 不同, VAE 输入数据, encoder 输出数据分布



- 我们有 encoder 输出的数据分布 $q_{\theta}(z|x)$ 的方差和均值, 于是可以使用从 θ 参数化的分布 $q_{\theta}(z|x)$ 中采样编码 z
- 编码 z 此后会被输入到 Φ 参数化的 decoder 中, 其输出解码的分布 $p_{\Phi}(x|z)$, 于是我们可以通过该分布进行采样

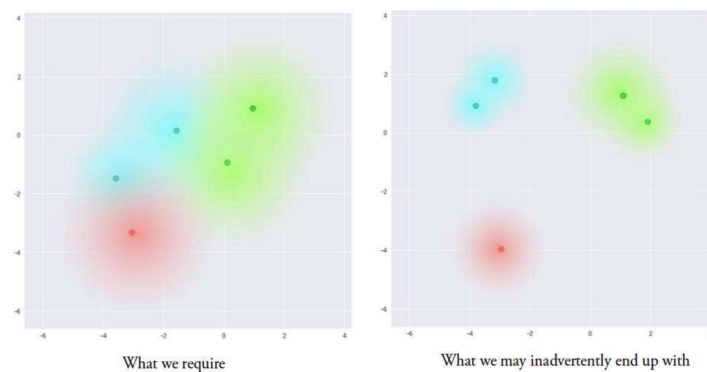


VAE 的损失

我们可以通过对数似然 $\log p_\phi(x|z)$ 来衡量解码器学习从 z 中重构 x 的效果。但是从神经网络的视角，我们需要一个损失函数来反向传播以学习最佳的参数。

$$l_{i(\theta, \varphi)} = -\mathbb{E}_{z \sim q_\theta(z|x_i)} [\log p_\varphi(x_i | z)] + \text{KL}(q_\theta(z | x_i) \parallel p(z))$$

- 这是单数据点的损失，总损失为所有的 l_i
- 注意到我们采样许多编码 z ，于是便可以使用期望似然估计 $-\mathbb{E}_{z \sim q_\theta(z|x_i)} [\log p_\varphi(x_i | z)]$ 来鼓励解码器学习重构数据
- 正则项 $\text{KL}(q_\theta(z | x_i) \parallel p(z))$ 衡量 q 和 p 分布的接近程度，在 VAE 中我们让 $p(z) = \text{Gaussian}(0, 1)$ 。这一项可以让编码器学习数据空间中的连续分布，不会让数据映射到空间中的不同的区域，基于流形假设使得该空间变得有意义



概率论视角理解 VAE

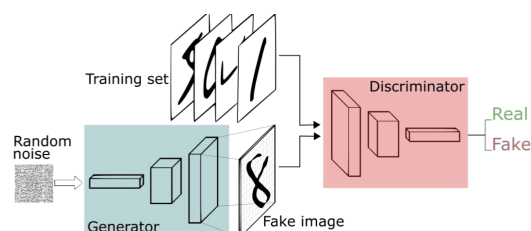
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Generative Adversarial Networks

动机：VAE 可以通过同时学习 encoder 和 decoder 来生成数据，那么可以只学习 generator 来生成数据吗？我们希望学习分布 p_{model} 以匹配 p_{data} 。

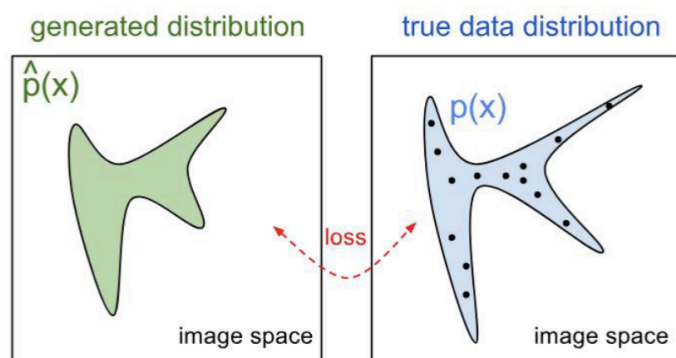
GAN

- Generator: $G(z)$ 输入随机噪声 z ，输出一个模仿训练数据分布的图像
- Discriminator: $D(x)$ 输入图片 x ，判断是图片是真/假的概率



对抗训练：生成器尝试去欺骗判别器，判别器尝试更好地区别假的和真的图像。训练结束时，判别器应该无法判断生成器的数据是否是真的还是假的，也就是说生成器生成的数据分布应该要与训练数据接近。

GAN 的损失



训练 G 和 D 遵循最小化最大值的策略

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_{z(z)}} [\log(1 - D(G(z)))]$$

- $\mathbb{E}_{z \sim p_{z(z)}} [\log(1 - D(G(z)))]$ 项为 D 输入为 G 从随机噪声分布生成的数据的先验下，判别器判断是 G 生成的虚假数据的对数似然
- $\mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)]$ 项为 D 输入为真实数据的先验下，判别器判断是真实数据的对数似然
- D 需要最大化正确判断输入数据来源的概率， G 需要最小化被识别的概率，最后达到纳什均衡

GAN 的训练

Gradient ascent on D :

$$D^* = \arg \max_D V(G, D)$$

Gradient descent on G

$$\begin{aligned} G^* &= \arg \min_G V(G, D) \\ &= \arg \min_G \mathbb{E}_{z \sim p} \log(1 - D(G(z))) \end{aligned}$$

实际中我们使用梯度上升的方法，即最大化欺骗 D 的对数似然

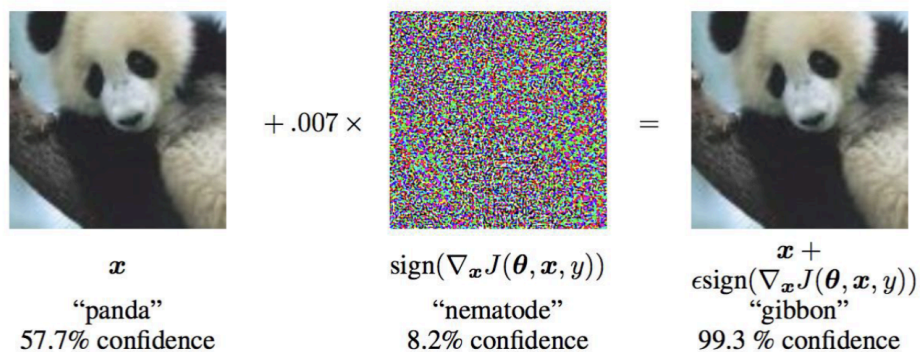
$$G = \arg \max_G \mathbb{E}_{z \sim p} \log(D(G(z)))$$

Adversarial Examples

Fast Gradient Sign Method

- 寻找相对于正确分类的梯度的符号，然后在输入数据上添加一个扰动

$$x \leftarrow x + \varepsilon \text{sign}(\nabla_x L(x, y))$$



Iterative GSM

- 通过多次添加扰动，可以更好地欺骗模型



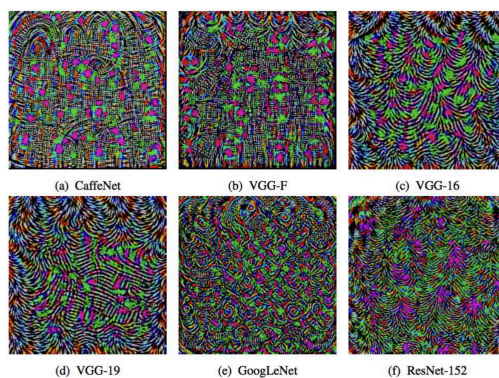
Least Likely Class Method

- 通过寻找相对于最不可能的类别的梯度的符号，可以更好地欺骗模型

$$x \leftarrow x - \epsilon \text{sign}(\nabla_x L(x, y))$$

Universal Adversarial Perturbations

For a given network, find an image-independent perturbation vector that causes all images to be misclassified with high probability.



Black-box Adversarial Examples

- 假设对抗只能通过输入输出来进行，而不能直接访问模型参数
- 通过生成一个与原始模型相似的模型，然后生成对抗样本

Properties of Adversarial Examples

- 对于任何的输入图像，都可以找到一个对抗样本使得模型产生错误的预测
- 获得对抗样本不需要精确的模型参数梯度
- 对抗样本有的时候可以经过变换也能欺骗模型（例如打印和拍照）
- 可以使用相同的扰动来欺骗不同的模型
- 能欺骗一个网络的对抗样本也有较高的概率能欺骗另一个参数甚至结构不同的网络

Why AE Works?

- 网络过于线性：结果可以预测
- 输入维度过高，所以可以对输入进行很多的微小变换
- 神经网络可以拟合任何函数，但是没有限制训练样本。反直觉的是，网络既能很好地概括自然图像，又容易受到对抗性示例的影响。

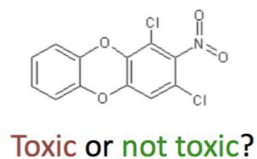
Defense Against Adversarial Examples

- Adversarial training：通过训练模型来生成对抗样本，然后将其添加到训练集中
- 学习如何拒绝对抗样本
- 设计高度非线性的结构鲁棒地对抗对抗样本
- 数据预处理

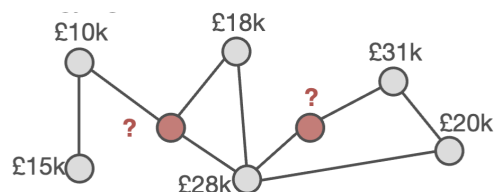
Graph Neural Networks

图网络可以用于

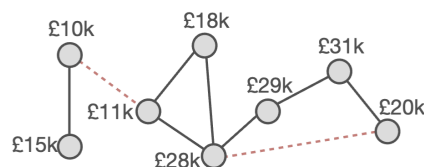
- Graph classification



- Node classification



- Link prediction



Learning on Graphs

- 通过将图数据转换为矩阵（向量），然后使用传统的神经网络来处理
- 但是这不是一个简单的任务，我们需要考虑到图的结构信息

Graph Convolutional Networks

假设对于图 G 每一个节点 i 都有一个特征向量 x_i ，邻接矩阵 A 。

- 输入 节点的特征矩阵 X (或 H) 和邻接矩阵 A
- 输出 一个矩阵 Z ，与 G 的每一个节点对应

网络的每一层对他们的输入进行非线性变换

$$H^{(l+1)} = f(H^{(l)}, A)$$

A Simple GCN

$$H^{(l+1)} = \text{ReLU}(AH^{(l)}W^{(l)})$$

- 某个节点的特征有聚合到新的节点特征矩阵中
 - 可以通过添加每一个节点的自身特征来保留节点的特征
- 不同的节点有不同的度，较大的度可能有较大的表示，反之亦然，可能导致梯度爆炸或者梯度消失
 - 可以通过标准化邻接矩阵来让特征向量的尺度保持一致

$$H^{(l+1)} = \text{ReLU}(D^{-1}AH^{(l)}W^{(l)})$$

- 我们如果不只考虑 i 节点的度的，还考虑 j 时可以

$$H^{(l+1)} = \text{ReLU}(D^{(-\frac{1}{2})}AD^{(-\frac{1}{2})}H^{(l)}W^{(l)})$$